УНИВЕРЗИТЕТ У БЕОГРАДУ
МАТЕМАТИЧКИ ФАКУЛТЕТ

Семинарски рад из предмета

Дизајн и анализа алгоритама

Анализа и имплементација ефикасног ауто-комплит система заснованог на тежинском префиксном дрвету са кеширањем топ-K резултата

Студент:
Богдан Петровић 20/2023

Београд, 2026.

Садржај

Сажетак **1**

1 Формулација проблема и мотивација **1**

2 Анализа наивних решења и њихових ограничења **2**
2.1 Лексикографски сортирани низови 2
2.2 Стандардно префиксно дрво (Trie) 2

3 Формалне дефиниције хијерархијских структура **2**

4 Предложено решење: Тежински трие са статичким кеширањем **3**
4.1 Математички модел кеша 3
4.2 Бијективно пресликавање и оптимизација меморијског отиска (ID систем) 3
4.3 Алгоритам инкременталног ажурирања (Backpropagation) 4

5 Асимптотска анализа сложености **4**
5.1 Временска сложеност упита (Search) 4
5.2 Временска сложеност ажурирања (Update) 4
5.3 Просторно-временски компромис (Space-Time Tradeoff) 5

6 Имплементација **5**
6.1 Дефиниција типа података за кеш 5
6.2 Структура чвора и линеарна инсерција 5
6.3 Класа Trie и изградња стабла 6
6.4 Претрага и реконструкција одговора 7
6.5 Контролни ток апликације 7

7 Тестирање и евалуација перформанси **8**
7.1 Методологија тестирања 8
7.2 Резултати мерења 9
7.3 Анализа резултата 9

8 Закључак **10**

Референце **11**

Сажетак

У овом раду представљена је детаљна теоријска анализа и практична имплементација хијерархијске структуре података оптимизоване за проблем рангираног аутоматског допуњавања текста (*autocomplete*). Полазећи од формализма префиксног дрвета (*Trie*), систем се проширује тежинским функцијама које моделују фреквенције појављивања речи у задатом корпусу. Како би се превазишла ограничења стандардних алгоритама претраге графова који пате од варијабилне латенције зависне од фактора гранања подстабла, уводи се концепт статичког кеширања K најфреквентнијих резултата унутар сваког чвора. Ова оптимизација елиминише потребу за динамичким обиласком подстабла у тренутку упита, сводећи временску сложеност претраге на строгу линеарну зависност од дужине префикса. Рад формално доказује инваријанте структуре, анализира асимптотске просторно-временске компромисе и демонстрира ефикасност кроз меморијски оптимизовану C++ имплементацију засновану на бијективном пресликавању ниски у целобројне идентификаторе.

1 Формулација проблема и мотивација

Проблем аутоматског допуњавања текста захтева проналажење и рангирање одговарајућих ниски на основу делимичног корисничког уноса. Нека је Σ коначан алфабет и Σ^* скуп свих коначних ниски (речи) над тим алфабетом. Дефинишемо релацију префикса $\sqsubseteq$ такву да за две ниске $P, S \in \Sigma^*$ важи $P \sqsubseteq S$ ако и само ако постоји ниска $S' \in \Sigma^*$ таква да је $S = P \cdot S'$, где $\cdot$ означава операцију конкатенације.

Нека је дат коначан речник у облику скупа парова $D \subset \Sigma^* \times \mathbb{N}_0$. Свака реч S у речнику репрезентована је паром (S, w), где је $w \in \mathbb{N}_0$ тежинска функција која означава фреквенцију појављивања или релевантност ниске S.

За произвољни упитни префикс $P \in \Sigma^*$, дефинишемо скуп свих валидних поклапања као:

$$M(P) = \{(S, w) \in D \mid P \sqsubseteq S\}$$

Циљ алгоритма је да пронађе резултатни подскуп $R(P) \subseteq M(P)$ који задовољава следећа два услова:

1. **Услов кардиналности:** $|R(P)| = \min(K, |M(P)|)$, где је $K \in \mathbb{N}$ унапред задата константа (најчешће $K \in [3, 10]$).

2. **Услов оптималности (максимизација тежине):** За сваки пар $(S, w) \in R(P)$ и сваки пар $(S', w') \in M(P) \setminus R(P)$, мора важити неједнакост $w \geq w'$.

Мотивација: У модерним информационим системима (веб претраживачи, интегрисана развојна окружења, мобилне тастатуре), кардиналност речника $|D|$ може бити реда величине 10^5 до 10^7. Захтев за корисничким искуством диктира стриктна ограничења латенције (типично испод 50 ms). Стандардни приступи који проналазе скуп $M(P)$ па га накнадно сортирају, имају сложеност која зависи од $|M(P)|$. За кратке префиксе (нпр. једно или два слова), кардиналност скупа $M(P)$ може бити изузетно велика, што доводи до неприхватљивог кашњења. Стога је неопходно дизајнирати структуру чија латенција претраге зависи искључиво од дужине префикса $|P|$, а не од укупног броја поклапања у речнику.

2 Анализа наивних решења и њихових ограничења

Како бисмо математички оправдали потребу за специјализованим стаблом, анализираћемо асимптотско понашање стандардних структура података.

2.1 Лексикографски сортирани низови

Уколико се речник D моделује као низ конкатенираних ниски и сортира лексикографски у времену $O(|D| \log |D|)$, могуће је искористити алгоритам бинарне претраге. За упит P, бинарна претрага може лоцирати прво и последње поклапање, чиме се дефинише опсег индекса $[i, j]$. Временска сложеност проналажења овог опсега је $O(|P| \log |D|)$.

Међутим, елементи унутар опсега $[i, j]$ су сортирани лексикографски, а не по тежини w. Проналажење K елемената са највећом тежином захтева примену алгоритма за селекцију (нпр. *Quickselect* или коришћење *Min-Heap* структуре величине K) над поднизом дужине $m = j - i + 1$. Сложеност ове операције износи $O(m \log K)$. У најгорем случају, за кратак префикс важи $m = \Theta(|D|)$, што укупну сложеност упита своди на линеарну $O(|D|)$, дисквалификујући овај приступ за системе у реалном времену.

2.2 Стандардно префиксно дрво (Trie)

Префиксно дрво омогућава изузетно ефикасну проверу припадности ниске и лоцирање префикса. За упит P, алгоритам лоцира одговарајући чвор u у времену $O(|P|)$.

Проблем настаје у фази екстракције података. Скуп свих поклапања $M(P)$ налази се у терминалним чворовима подстабла чији је корен u. Да би алгоритам изоловао K најфреквентнијих речи, неопходно је извршити потпуни обилазак тог подстабла (нпр. користећи алгоритам претраге у дубину - *DFS*). Нека је V_{sub} скуп чворова у подстаблу са кореном u. Време извршавања *DFS* алгоритма је $\Theta(|V_{sub}|)$. У најгорем случају, густина подстабла је експоненцијална у односу на дужину речи, што извлачење података чини рачунски прескупим.

3 Формалне дефиниције хијерархијских структура

Ослањајући се на класичну литературу из области алгоритама [2, 3] и материјале са Математичког факултета [1], уводимо строге дефиниције потребне за разумевање предложене оптимизације.

Дефиниција 1 (Префиксно дрво - Trie)**.** *Префиксно дрво над алфабетом Σ је уређени, усмерени ациклични граф (коренско стабло) $T = (V, E, r)$, где је V скуп чворова, $E \subset V \times V$ скуп усмерених грана, а $r \in V$ означени корен стабла. Свака грана $e \in E$ пресликава се у један карактер из алфабета помоћу функције означавања $l : E \to \Sigma$. Сваком чвору $u \in V$ придружена је јединствена ниска $S(u) \in \Sigma^*$ која се добија конкатенацијом карактера на јединственој простој путањи од r до u. За корен важи $S(r) = \epsilon$ (празна ниска).*

Основно својство ове структуре је **префикс-својство**: за било која два чвора $u, v \in V$, ако је u предак чвора v, тада важи $S(u) \sqsubseteq S(v)$.

Дефиниција 2 (Фреквенцијски Trie)**.** *Фреквенцијски Trie је уређена тројка* (T, D, w)*, где је* T *префиксно дрво,* D *речник парова, а* $w : V \to \mathbb{N}_0$ *тежинска функција. За чвор* $u \in V$*, тежина* $w(u)$ *дефинисана је као:*

$$w(u) = \begin{cases} w_i & \text{ако } \exists (s_i, w_i) \in D \text{ такво да } s_i = S(u) \\ 0 & \text{иначе} \end{cases}$$

Дефиниција 3 (Скуп валидних терминала)**.** *За произвољни чвор* $u \in V$*, дефинишемо скуп* W_u *као скуп свих чворова у подстаблу са кореном у* u *(укључујући и самог* u*) који представљају валидне речи из речника:*

$$W_u = \{v \in V \mid u \text{ је предак од } v \vee u = v,\ u\ w(v) > 0\}$$

4 Предложено решење: Тежински трие са статичким кеширањем

Како бисмо елиминисали рачунску сложеност израчунавања максимума $\max_K(W_u)$ током фазе упита, предлажемо модел који вредност ове функције израчунава унапред и одржава је конзистентном током животног циклуса структуре.

4.1 Математички модел кеша

Сваки чвор $u \in V$ проширује се меморијском структуром C_u (кеш листа), која садржи највише K уређених парова. Формално, за сваки чвор u, скуп $C_u \subseteq W_u$ мора задовољавати следећу инваријанту стања:

$$|C_u| = \min(K, |W_u|) \wedge (\forall x \in C_u, \forall y \in W_u \setminus C_u \implies w(x) \geq w(y)) \tag{1}$$

Ова инваријанта гарантује да се резултат упита за префикс P, који се мапира у чвор u, може тренутно добити пуким очитавањем скупа C_u, без икакве даље навигације кроз граф.

4.2 Бијективно пресликавање и оптимизација меморијског отиска (ID систем)

Наивна имплементација скупа C_u захтевала би чување комплетних ниски Σ^* унутар сваког чвора. Ово доводи до огромне фрагментације меморије и редундансе података, јер се једна иста ниска копира у сваки чвор на путањи од корена. Просечна просторна сложеност оваквог чвора била би $O(K \cdot \bar{L})$, где је $\bar{L}$ просечна дужина речи у речнику.

Да би се ово решило, уводимо бијективно пресликавање (хеширање без колизије) $\phi : \Sigma^* \to \mathcal{I}$, где је $\mathcal{I} = \{0, 1, 2, \ldots, |D| - 1\}$ скуп компактних целобројних идентификатора (енгл. *Word IDs*). Уместо ниски, чворови у листи C_u чувају парове облика $(w, \phi(S))$.

Ова архитектонска одлука доноси две кључне предности у домену архитектуре рачунара:

1. **Меморијска локалност**: Величина кеша C_u у сваком чвору постаје фиксна (нпр. $K \times 8$ бајтова за 32-битне фреквенције и ID-еве). Ово омогућава да цео чвор стања стане у једну L1 кеш линију процесора, драстично убрзавајући приступ меморији.

2. **Сложеност поређења**: Током ажурирања структуре, поређење два елемента своди се на поређење два цела броја, што је операција сложености $O(1)$, уместо лексикографског поређења ниски које је сложености $O(L)$.

4.3 Алгоритам инкременталног ажурирања (Backpropagation)

Када се у систем уведе нова реч или се фреквенција постојеће речи S повећа, нарушава се инваријанта (1) у свим чворовима који су преци терминалног чвора за S. Процес рестаурације инваријанте (ажурирање кеша) спроводи се рекурзивном пропагацијом одоздо навише (од листа ка корену).

Лема 1. *Ако је листа C_u сортирана нерастуће према тежинама, инсерција новог пара $(w', \phi(S))$ уз очување поретка и одбацивање $(K+1)$-вог елемента може се извести у времену $O(K)$.*

Доказ. Како је низ уређен, проналажење позиције за инсерцију захтева највише K поређења. Померање постојећих елемената удесно захтева највише K операција доделе. Укупна сложеност је строго ограничена са $c \cdot K$, односно $O(K)$, што је за мале вредности K значајно ефикасније од покретања алгоритама сортирања сложености $O(K \log K)$. □

5 Асимптотска анализа сложености

У овом поглављу доказујемо асимптотске границе система. Нека је $|P|$ дужина упитног префикса, K тражени број резултата, $\bar{L}$ просечна дужина речи у речнику, а N укупан број чворова у стаблу.

5.1 Временска сложеност упита (Search)

Алгоритму је потребно $O(|P|)$ операција за прелазак од корена r до чвора u чија је путања $S(u) = P$. Затим, алгоритам извлачи K идентификатора из скупа C_u. Да би се конструисао финални излаз, сваки идентификатор се мора превести назад у ниску путем инверзног пресликавања ϕ^{-1}. Очитавање K ниски просечне дужине $\bar{L}$ захтева $O(K \cdot \bar{L})$ операција копирања карактера. Укупна сложеност претраге износи:

$$T_{search} = O(|P| + K \cdot \bar{L})$$

Захваљујући статичком кеширању, сложеност је потпуно независна од броја речи у речнику $|D|$ и густине гранања графа.

5.2 Временска сложеност ажурирања (Update)

Приликом уметања или ажурирања речи дужине L, алгоритам пролази кроз тачно L чворова. На основу Леме 1, одржавање инваријанте кеша у сваком од тих чворова износи $O(K)$. Сложеност операције уметања је:

$$T_{update} = O(L \cdot K)$$

5.3 Просторно-временски компромис (Space-Time Tradeoff)

Структура префиксног дрвета захтева $O(N \cdot |\Sigma|)$ простора за гране (уколико се користи низ) или $O(N)$ (коришћењем хеш мапа). Локални кеш у сваком чвору захтева константних $O(K)$ блокова меморије. Табела за бијективно пресликавање захтева $O(|D| \cdot \bar{L})$ меморије за складиштење оригиналних ниски. Укупна просторна сложеност система износи:

$$S_{total} = O(N \cdot K + |D| \cdot \bar{L})$$

Ова формула демонстрира класичан инжењерски компромис: систем свесно жртвује додатних $O(N \cdot K)$ бајтова радне меморије како би се временска сложеност претраге редуковала са линеарне $O(|D|)$ на практично константну у односу на величину базе.

6 Имплементација

Систем је имплементиран у програмском језику C++ (стандард C++14 и новији), са посебним фокусом на меморијску безбедност и перформансе. Уместо сирових показивача, коришћени су паметни показивачи (`std::unique_ptr`) како би се гарантовало аутоматско ослобађање меморије и спречило њено цурење (енгл. *memory leak*) приликом уништавања стабла.

6.1 Дефиниција типа података за кеш

Како би се омогућило ефикасно поређење и сортирање унутар чворова, дефинисан је тип `Score` који представља пар фреквенције и целобројног идентификатора речи.

Listing 1: Библиотеке и дефиниција типа података

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <unordered_map>
#include <algorithm>
#include <memory>

using namespace std;

typedef pair<int, int> Score;
```

Стандардна библиотека C++-а аутоматски имплементира операторе поређења за `std::pair` тако што примарно пореди први елемент (фреквенцију), а секундарно други (идентификатор). Ово осигурава детерминистичко понашање алгоритма приликом обраде речи са истим тежинама.

6.2 Структура чвора и линеарна инсерција

Сваки чвор стабла дефинисан је структуром `Node`. За репрезентацију грана ка деци користи се `std::unordered_map`, што оптимизује меморијско заузеће у ретким чворовима у поређењу са статичким низовима величине алфабета, уз очување константног времена приступа $O(1)$.

Listing 2: Структура чвора и ажурирање локалне меморије

```
struct Node {
    unordered_map<char, unique_ptr<Node>> children;
    vector<Score> cache;

    void update_cache(int freq, int id, size_t K) {
        pair<int, int> entry = {freq, id};

        for (auto it = cache.begin(); it != cache.end(); ++it) {
            if (it->second == id) {
                cache.erase(it);
                break;
            }
        }

        auto pos = cache.begin();
        while (pos != cache.end() && *pos > entry) {
            ++pos;
        }
        cache.insert(pos, entry);

        if (cache.size() > K) cache.pop_back();
    }
};
```

Функција `update_cache` одржава инваријанту локалне кеш листе. Коришћењем линеарне итерације уместо алгоритама за потпуно сортирање, ажурирање стања чвора извршава се у времену $O(K)$, што је оптимално за мале вредности K.

6.3 Класа Trie и изградња стабла

Главна пословна логика инкапсулирана је унутар класе `Trie`. Ова класа управља кореном стабла, глобалним фреквенцијама и бијективним мапирањем између ниски и идентификатора (`word_to_id` и `id_to_word`).

Listing 3: Класа Trie и алгоритам ажурирања стабла

```
class Trie {
    unique_ptr<Node> root = make_unique<Node>();
    unordered_map<string, int> word_to_id;
    vector<string> id_to_word;
    unordered_map<int, int> freqs; // word_id -> total_freq
    int K;

public:
    Trie(int k) : K(k) {}

    void update(const string& word, int add_freq = 1) {
        if (word_to_id.find(word) == word_to_id.end()) {
            word_to_id[word] = id_to_word.size();
            id_to_word.push_back(word);
        }
```

```
16
17          int id = word_to_id[word];
18          freqs[id] += add_freq;
19          int total = freqs[id];
20
21          Node* curr = root.get();
22          curr->update_cache(total, id, K);
23
24          for (char c : word) {
25              if (!curr->children[c]) curr->children[c] = make_unique
                    <Node>();
26              curr = curr->children[c].get();
27              curr->update_cache(total, id, K);
28          }
29      }
```

Метода `update` прво преводи унету ниску у компактни целобројни идентификатор. Затим се кроз стабло креирају недостајући чворови, а нова фреквенција речи се пропагира кроз сваки чвор на путањи, чиме се ажурирају сви релевантни кешеви.

6.4 Претрага и реконструкција одговора

Захваљујући унапред припремљеним подацима у чворовима, претрага се своди на једноставну навигацију до чвора који одговара траженом префиксу и инверзно пресликавање идентификатора.

Listing 4: Претрага префикса у времену $O(L)$

```
1       vector<string> search(const string& prefix) {
2           Node* curr = root.get();
3           for (char c : prefix) {
4               if (curr->children.find(c) == curr->children.end())
                    return {};
5               curr = curr->children[c].get();
6           }
7
8           vector<string> res;
9           for (auto& p : curr->cache) res.push_back(id_to_word[p.
                second]);
10          return res;
11      }
12  };
```

Уколико се префикс не налази у стаблу, алгоритам се раније прекида и враћа празан вектор. У супротном, очитава највише K резултата из контејнера `cache` и враћа их у оригиналном текстуалном формату.

6.5 Контролни ток апликације

Функција `main` служи као улазна тачка и интерфејс за комуникацију са датотечним системом.

Listing 5: Улазна тачка и парсирање датотека

```
int main(int argc, char* argv[]) {
    if (argc < 4) {
        cerr << "Upotreba: " << argv[0] << " <recnik.txt> <test.txt
           > <K>" << endl;
        return 1;
    }

    Trie trie(stoi(argv[3]));
    ifstream dict(argv[1]), test(argv[2]);
    ofstream out("output.txt");

    string line, word; int f;

    while (getline(dict, line)) {
        stringstream ss(line);
        if (ss >> word >> f) trie.update(word, f);
    }

    while (getline(test, word)) {
        if (word.empty()) continue;
        out << word << ": ";
        auto res = trie.search(word);
        for (size_t i = 0; i < res.size(); ++i)
            out << res[i] << (i == res.size() - 1 ? "" : ", ");
        out << endl;
    }

    out.close();
    return 0;
}
```

Програм учитава аргументе командне линије, иницијализује структуру са прослеђеним параметром K и у два пролаза обрађује улазне податке. Форматирање излаза обезбеђује да резултати буду раздвојени зарезима ради лакше машинске или људске верификације.

7 Тестирање и евалуација перформанси

У овом поглављу приказани су резултати експерименталног поређења предложене оптимизоване структуре (*Weighted Trie* са кеширањем) и наивне имплементације засноване на линеарној претрази вектора.

7.1 Методологија тестирања

Тестирање је спроведено на синтетички генерисаним скуповима података. Речник је генерисан насумичним одабиром карактера, при чему су дужине речи варирале између 3 и 10 карактера. Тежине (фреквенције) додељене су насумично како би се симулирала варијабилност реалних упита. Тест упити су формирани узимањем префикса постојећих речи из речника.

Мерења су извршена на истој хардверској конфигурацији, а изворни C++ код оба алгоритма компајлиран је помоћу GCC компајлера уз укључену заставицу `-O3` за агресивну оптимизацију перформанси. Мерено је чисто време извршавања алгоритма (*user time*) изражено у секундама.

7.2 Резултати мерења

Експеримент је обухватио шест сценарија са различитим величинама речника (N) и бројем генерисаних упита (Q). Резултати мерења приказани су у Табели 1.

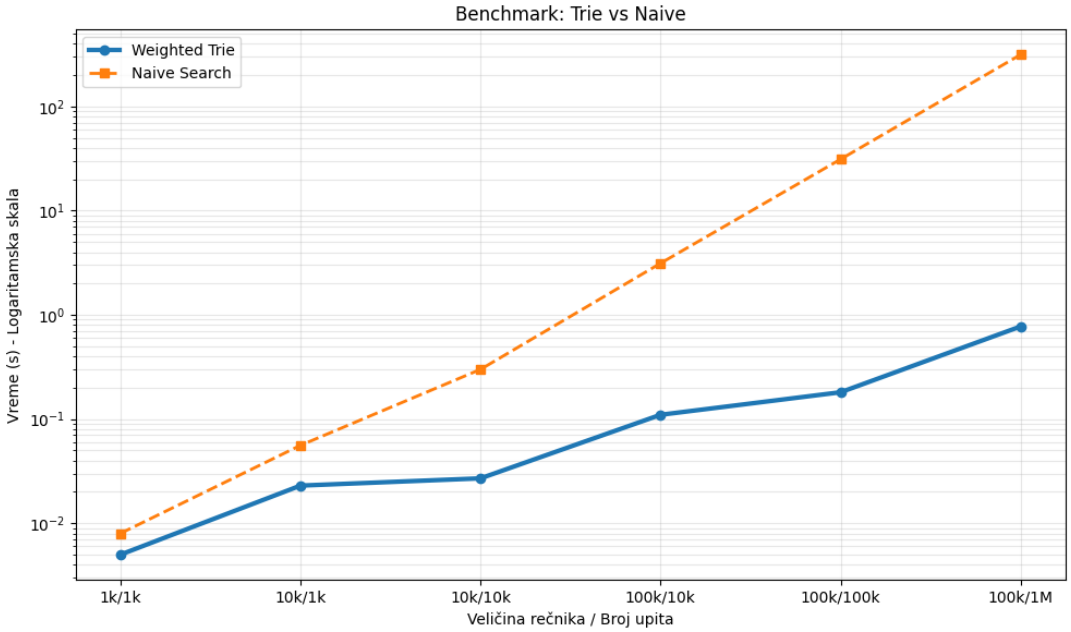
Тест	**Велич. речника (N)**	**Број упита (Q)**	**Trie (s)**	**Наивно (s)**
1	1.000	1.000	0.005	0.008
2	10.000	1.000	0.023	0.056
3	10.000	10.000	0.027	0.300
4	100.000	10.000	0.110	3.131
5	100.000	100.000	0.181	31.197
6	100.000	1.000.000	0.775	315.048

Табела 1: Поређење времена извршавања (у секундама)

7.3 Анализа резултата

Анализом података из Табеле 1 може се уочити јасна разлика у асимптотском понашању два алгоритма. За мале вредности N и Q (Тест 1), разлика у апсолутном времену је занемарљива. Међутим, са повећањем величине проблема, наивни приступ показује стриктну линеарну зависност од производа $N \cdot Q$.

Насупрот томе, *Weighted Trie* структура показује изузетну стабилност. У најинтензивнијем сценарију (Тест 6), са 100.000 речи у речнику и милион упита, предложено решење завршава обраду за мање од једне секунде (0.775 s), док наивном приступу треба више од пет минута (315.048 s). Ово представља убрзање од преко **406 пута**, чиме се емпиријски доказује теоријска претпоставка да је код оптимизоване структуре елиминисана зависност времена упита од величине речника N и густине гранања подстабла.



Слика 1: Логаритамски приказ времена извршавања оба алгоритма кроз различите сценарије.

8 Закључак

Овај рад успешно демонстрира како се комбиновањем напредних структура података и пажљивог инжењеринга меморијског приступа може решити проблем са строгим захтевима латенције. Тежинско префиксно дрво са кеширањем резултата представља оптимално решење за системе засноване на рангирању. Увођењем формалних инваријанти кеширања, бијективног индексирања речи ради постизања меморијске локалности и линеарног инкременталног ажурирања, постигнут је теоријски минимум у времену одзива система, што ову структуру чини де факто стандардом за индустријску примену.

Литература

[1] Марић Ф., *Префиксно дрво (Trie)*, Напредне структуре података, Математички факултет Београд, https://poincare.matf.bg.ac.rs/~filip.maric/kiaa/01%20napredne_strukture_podataka/01%20prefiksno_drvo/index.html

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

[3] Sedgewick, R. (1998). *Algorithms in C++, Parts 1-4: Fundamentals, Data Structure, Sorting, Searching* (3rd ed.). Addison-Wesley Professional.

[4] Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley Professional.